Cache Me if You Can: Repairing the KV State after Compression

Anonymous Authors[1]

Abstract

As large language models (LLMs) support longer contexts and more capable agentic sessions, what matters in the accumulated context can change with each new query, tool output, test result, or browser observation. At the same time, inference still depends on a limited active key-value (KV) cache, and many existing KV-cache compression methods decide which past tokens stay active from the current prefix. As a result, tokens that matter in later turns may already have been evicted or compressed away in earlier compression decisions. We study post-compression KV cache repair, a runtime mechanism that restores previously evicted tokens to the active cache. REPAIRKV uses the interval between turns to reevaluate which tokens are important from evicted KV rows stored in a slower memory tier, then promotes a small subset before decoding resumes. Controlled experiments show the same repair effect under changing relevance and different initial eviction policies (compression styles), and a preliminary diagnostic on open-source repositories shows the same effect; on Qwen2.5-7B-Instruct at 32K context, REPAIRKV achieves 91.0% retrieval on a four-query needle-in-a-haystack task versus 24.5% for the matched no-repair baseline at the same active-cache budget, with only 96 promoted tokens.

1. Introduction

Motivation. Long-context inference increasingly depends on KV-cache compression and offload. During decoding, a transformer stores key-value (KV) activations for prior tokens so later tokens can attend to them. The model weights are fixed once loaded, but the KV cache grows with the number of tokens kept for each live sequence; at long context lengths, this sequence-dependent state can dominate available GPU memory and increase inference-time attention and cache-management costs. KV-cache compression and offload reduce this pressure by deciding which historical state remains active on GPU and which state is evicted or moved to a slower tier. That decision is usually made with information available at the current turn, so it is often treated as a one-way operation: evict now, decode later from the retained active set.

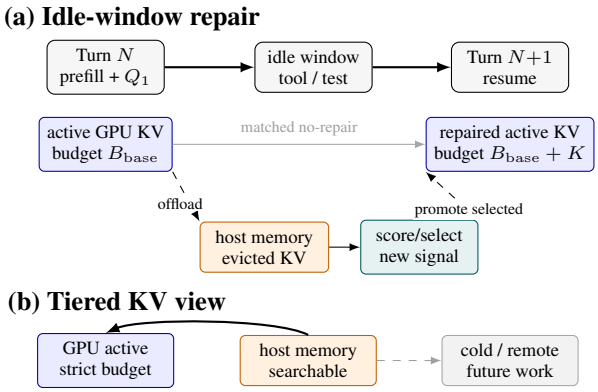


Figure 1. REPAIRKV system overview: a paused-request KV-state adaptation primitive. A paused request offloads evicted KV to host memory, scores and promotes a budgeted subset after the next-turn signal arrives, and exposes a conceptual tiered-memory interface that production engines would materialize through page or block allocators.

This static view misses two facts about multi-turn inference. First, attention relevance is query- and context-dependent: later turns, retrieved documents, tool outputs, test failures, or file changes can make previously low-priority context newly important. Query-aware and fine-grained KV retrieval systems show that both token criticality and attention-distribution patterns can change with the active query (Tang et al., 2024; Wang et al., 2025). Second, agentic workflows often contain non-decoding intervals while the model waits for external results. Recent systems make this operationally visible: Continuum schedules multi-turn agent requests around tool-call gaps and KV-cache lifetimes, while AgentServe targets local agent serving under tight GPU memory budgets (Li et al., 2026; Zhang et al., 2026). These systems show that turn boundaries, cache lifetime, and tool-call waits are now explicit systems objects, but they preserve

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

or schedule cached state rather than repairing an already-compressed active cache using the next-turn signal.

Coding-agent measurements reinforce why this interval is worth studying: OS-level execution, including tool calls and setup, can account for 56–74% of task latency (Zheng et al., 2026). Furthermore, modern production serving stacks such as vLLM (Kwon et al., 2023) now deploy prefix caching by default, but re-deriving evicted KV through a fresh prefill pass remains the fallback on cache miss, eviction, or paused-request discard. Prior work measures this cost at up to 99% of multi-turn prefill time when historical KV is repeatedly recomputed (Gao et al., 2024) and at 37% of end-to-end execution time when paused-request KV is discarded in agentic workloads (Abhyankar et al., 2024), motivating tiered KV substrates that trade recompute for offloaded reuse (Liu et al., 2025; Dzikanyanga et al., 2026). Together, changing relevance, non-decoding slack, and the persistent cost of recompute create the opportunity we study: use the pause before the next decode to revise active KV state after compression.

This paper studies whether that pause can improve the state seen by the next decode. In this setting, compression is not only a storage-saving step. It also decides which past tokens remain immediately available to the next decode. Once a request has paused, the runtime may hold a compressed active cache plus a host-memory store (CPU DRAM) of evicted KV rows. If the next-turn cue becomes available before generation resumes, the system has an opportunity to revise which prior context is active. In shared serving this is a scheduler tradeoff; in the single-session or dedicated setting we target, the same interval is often an idle window for the paused request. Either way, the central question is the same: can newly revealed turn-$(N{+}1)$ information repair a compressed cache under the same resumed active-cache budget.

Setting. Recent analysis shows that KV compression can sharply degrade instructions and facts that later become important (Chen et al., 2025). The missing evaluation question is not simply whether compression loses information, or whether a serving system can preserve state across a pause. It is whether future-turn information can improve an already-compressed cache under the same resumed active-cache budget. A useful benchmark for this question needs explicit turn boundaries, a hidden future query, shared pre-turn history, and no-repair controls matched on resumed-cache budget $B_{\text{base}} + K$. In our experiments, the future relevance signal is an explicit second-turn (Q_2) retrieval query so that the repair effect is measurable, while the intended workflow claim is broader than query answering.

Contributions. REPAIRKV is an inference-time adaptation operator for tiered KV state, conditioned on the newly available next-turn signal. The adaptation is non-parametric: model weights, prompts, and decoding settings are fixed, while the active cache state is revised under a resource budget. The operator adds a reverse direction to the usual compression lifecycle: KV can be evicted or offloaded when memory is scarce, then selectively promoted when the relevance signal changes. After turn N, the runtime compresses the evictable context portion to an active set C_{base} with $|C_{\text{base}}| = B_{\text{base}}$ and retains evicted KV in host memory as W_N. When the turn-$(N{+}1)$ signal arrives, REPAIRKV scores that store and promotes a subset $S_K(Q_2) \subseteq W_N$, $|S_K| \leq K$, back into the active GPU cache before decoding resumes. This paper uses an explicit Q_2 retrieval query as the scoring signal because it gives a controlled first test of whether post-compression repair can exploit new relevance information. Unlike pause/resume serving, REPAIRKV changes the resumed active cache. Unlike within-turn query-aware compression, it starts from a paused compressed state and does not rerun full-prefix selection after Q_2 is revealed. We contribute:

- a matched resumed active-cache-budget protocol for the post-compression, pre-resume lifecycle stage, isolating cache repair from simply keeping more active tokens.
- calibrated evidence on pooled split-query multi-query needle-in-a-haystack (MQ-NIAH) 4Q/6Q/8Q evaluations, a five-turn revisit diagnostic, three first-stage eviction policies, a Scissorhands-style persistence stress test, and preliminary real-repository diagnostic evidence, where REPAIRKV exceeds matched no-repair and content-agnostic controls while the real-repo variants remain below a label-assisted locality reference.
- a repair prototype with transfer/promotion feasibility results and a controlled GPU runtime probe for chunked scan, top-K selection, and KV movement.

This controlled scope identifies the repair mechanism on one primary model and one synthetic task family, with targeted eviction-policy and real-repository checks. The adaptation-facing implication is that long-context agents may need between-turn KV state updates in addition to pre-generation pruning. Figure 1 gives a system overview.

2. Related Work

Recent KV-cache compression and retrieval methods such as SnapKV, StreamingLLM, H_2O, and QUEST define the current frontier for memory-efficient long-context inference. SnapKV, StreamingLLM, and H_2O decide which past tokens remain active under a fixed budget, while query-aware methods such as QUEST emphasize that token criticality depends on the active query (Li et al., 2024a; Xiao et al., 2024; Zhang et al., 2023; Tang et al., 2024). Long-context and KV-lifecycle benchmarks stress contexts from ordinary long documents to 100K-token regimes, showing that application-

visible context length does not remove the need for memory-management mechanisms (Bai et al., 2024; Zhang et al., 2024; Hsieh et al., 2024; Li et al., 2025). SCBench makes the KV-cache lifecycle itself a benchmark target, and ShadowKV shows that sparse KV reconstruction and offload can improve long-context serving throughput (Li et al., 2025; Sun et al., 2025). Fine-grained KV retrieval systems dynamically fetch query-relevant rows from larger stores during active inference (Tang et al., 2024; Wang et al., 2025; Qi et al., 2026); attention analyses likewise show that token importance can be query-dependent or recurrent rather than fixed once compressed (Wu et al., 2024; Zhang et al., 2025). These works motivate dynamic KV state, but they mostly make the cache dynamic in one direction: retain less, offload more, or retrieve rows during active inference. Two camps coexist here: token-eviction methods (SnapKV, H_2O, Scissorhands) discard rows destructively, while sparse-attention/retrieval methods (Quest, FIER, ShadowKV) keep the full cache and merely skip rows during attention. REPAIRKV occupies a third position: rows leave the active GPU cache but persist in host memory, so the active set can be revised between turns. REPAIRKV targets the complementary direction. It starts from an already compressed paused state, does not rerun full-prefix selection after Q_2 is revealed, and revises the active cache before decoding resumes under a matched resumed-cache budget.

Serving systems provide the substrate for this question. PagedAttention/vLLM improves GPU KV allocation and block management during serving (Kwon et al., 2023), while InferCept preserves paused requests to avoid recomputing prefixes when augmented generation resumes (Abhyankar et al., 2024). Agent and tool-use benchmarks make external actions part of the workload, including API calls, web interaction, repository edits, and interactive tasks (Qin et al., 2024; Zhou et al., 2024; Jimenez et al., 2024; Liu et al., 2024). Agent-serving and KV-tiering systems reason about resume prefills, offloading, cache reuse, and tiered storage across GPU, host memory, storage, and network tiers (Li et al., 2026; Zhang et al., 2026; Liu et al., 2025; Dzikanyanga et al., 2026; Gao et al., 2024). These systems manage where KV state lives and how it is reused. InferCept's preserve strategy parallels our W_N store and its interception aligns with our pause boundary; CachedAttention's pre-loading is the closest existing analogue to our content-aware promote step. REPAIRKV adds a content-aware repair operator on top of the system-level preservation and pre-loading these works already provide, asking which offloaded cached units should re-enter the active cache when a later turn reveals a new relevance signal.

3. Method

Overview. Each compression step can only choose the active KV set with the information available at that point. Before a repaired cache is used for decoding, any available signal about the upcoming computation can guide which offloaded KV rows should re-enter the active cache. We study a minimal controlled repair mechanism: promote selected offloaded KV rows back into the active cache. Figure 1 summarizes the system overview. At the pause boundary, the abstraction has three objects:

- C_{base}: active evictable KV, with $|C_{\text{base}}| = B_{\text{base}}$ (base active-cache budget);
- W_N: offloaded evicted KV, hidden from decoding unless promoted;
- s_{N+1}: the newly available turn signal, instantiated as Q_2.

The repair map chooses $S_K(s_{N+1}) \subseteq W_N$ with $|S_K| \leq K$, where K is the restore budget (the number of evicted KV rows promoted back into the active cache), and resumes from $C_{\text{repair}} = C_{\text{base}} \cup S_K$ at resumed active-cache budget $|C_{\text{repair}}| \leq B_{\text{base}} + K$.

Two-turn protocol. The protocol proceeds in two turns. The model first processes a long context and answers Q_1 from the full cache. After that answer, the runtime compresses only the evictable context portion to a base budget B_{base}, while preserving the exact Q_1 prompt and generated-answer tail. During the subsequent tool-call idle window, the turn-$(N{+}1)$ query Q_2 becomes known. REPAIRKV scores offloaded evicted KV against the active post-Q_1 cache and Q_2 signal, promotes a K-budgeted subset S_K, and answers Q_2 from C_{repair}. The data path has two movements: evicted KV is offloaded to a host-memory tier after Q_1, and selected KV is copied back before Q_2 decoding. Note that we set aside full-prefill recompute baselines from the primary comparison, since re-deriving KV from token IDs after Q_2 is revealed varies the prefill term rather than the resumed active-cache budget. Compute-matched recompute baselines are deferred to future work (Section 6). This isolates the adaptation operator for one paused request on one GPU under three controlled conditions: a single host-memory tier, the full Q_2 signal known before resumed decoding, and a pre-materialized evicted-KV store. Scheduling, remote tiering, and cache contention are separate systems questions, discussed as future directions (Section 6).

Repair operation. Intuitively, REPAIRKV asks which evicted KV rows the next-turn query Q_2 would attend to most and promotes a small budgeted subset back into the active cache before decoding resumes, packed into local neighborhoods around the top-ranked anchors. Algorithm 1 states the framework: turn-by-turn decoding and compres-

Algorithm 1 General REPAIRKV Execution Framework

Input: turn signals $\{s_N\}_{N=1}^{T}$, eviction policy $\mathcal{P}$, base budget B_{base}, restore budget K.
Initialize: active cache $\mathcal{C}$, evicted store $\mathcal{W} \leftarrow \emptyset$.
for $N = 1$ to T **do**
  $\text{ans}_N \leftarrow \text{DECODE}(\mathcal{C}, s_N)$ // Decoding
  $\mathcal{C}, \mathcal{W} \leftarrow \mathcal{P}(\mathcal{C}, \mathcal{W}, B_{\text{base}})$ // Compression
  s_{N+1} arrives // Idle window begins
  for $i \in \mathcal{W}$ **do**
    $a_i \leftarrow \text{SCORE}(i \mid \mathcal{C}, s_{N+1})$ // Scoring
  end for
  rank $\mathcal{W}$ by a_i // Ranking
  $S_K \leftarrow \text{SELECT}(\mathcal{W}, K)$, $|S_K| \leq K$ // Selection
  $\mathcal{C} \leftarrow \mathcal{C} \cup S_K$; $\mathcal{W} \leftarrow \mathcal{W} \setminus S_K$ // Repair
end for

sion are interleaved with an idle-window step that scores, ranks, and selects a K-budgeted subset of evicted rows to promote before the next decode. Two repair-specific choices, SCORE and SELECT, fully specify the prototype; in our prototype, $\mathcal{C} = C_{\text{base}}$ and $\mathcal{W} = W_N$.

For SCORE, post-RoPE Q_2 query tensors $\{q_{2,t}^{(\ell,h)}\}$ are matched against keys in the active and offloaded sets via

$$A^{(\ell,h)} = \text{softmax}\left(\frac{q_{2,:}^{(\ell,h)}\,[K_{C_{\text{base}}},\, K_{W_N}]^\top}{\sqrt{d_k}}\right), \quad (1)$$

restricted to columns in W_N, then aggregated as $a_i = \text{mean}_{\ell,h} \max_t A_{t,i}^{(\ell,h)}$. Because the softmax normalizes over the active and offloaded keys, a_i is the attention mass row i would receive given C_{base} rather than a raw inner product. Ties break by the per-token score from the turn-N first-stage compression, then by ascending position.

For SELECT, each top-ranked anchor i is expanded into a local burst $\{i-L, \ldots, i+R\} \cap (W_N \setminus S_K)$ under a strict $|S_K| \leq K$ budget; the right-biased $(L, R) = (2, 20)$ reflects the query supplying the anchor while answer-bearing values trail it. If anchor bursts do not exhaust K, remaining slots backfill from single ranked rows. Burst widths are fixed in this prototype.

The current prototype scores token rows, but the systems abstraction is promotion over cached units; a systems implementation would batch these promotions into exposed blocks or pages.

Matched-budget evaluation. All main claims are made against a no-repair baseline with the same resumed active-cache budget. A matched resumed-cache budget means that Q_2 decoding sees the same number of active evictable-context tokens in addition to the identical preserved Q_1 prompt and answer tail. It does not mean that the runtime stores the same amount of offloaded KV state. Let B_{match} denote the no-repair condition that applies the base eviction policy with context budget $B_{\text{base}} + K$ while preserving the same Q_1 prompt and answer tail. REPAIRKV decodes from a $B_{\text{base}} + K$ active-cache budget by starting from B_{base} retained rows and promoting K rows from the host-memory store in this prototype. The extra store bytes, scorer time, and transfer time are reported outside the matched active-cache budget. The comparison therefore measures active-cache fairness; total service cost is reported separately. The main restore-budget frontier reports raw answer score so readers can compare absolute quality across task difficulty levels. For effect-size comparisons, we also report score gain over matched no-repair,

$$\Delta_{\text{repair}} = \text{Score}(\text{RepairKV}) - \text{Score}(B_{\text{match}}) \quad (2)$$

where Score is the mean fraction of held-out Q_2 targets recovered correctly per example, averaged over the fixed evaluation set shared across all conditions for that setting. This retrieval score measures target recall, while unconstrained answer quality is outside the controlled benchmark. Stricter exact-set and precision-sensitive scoring serve as confirmatory checks. For diagnostic purposes only, SpanRef-K uses benchmark metadata: it enumerates annotated Q_2 answer-span groups and selects the highest-scoring subset whose token cost is at most K. SpanRef-K is exact over this small span-group family. Broader K-token cache-state optima and upper bounds remain outside this diagnostic. Thus the protocol tests whether newly revealed turn-$(N{+}1)$ information can improve a compressed cache relative to matched no-repair eviction under the same active-cache budget.

4. Experimental Setup

Benchmark family. The main evaluation uses Qwen2.5-7B-Instruct (Qwen, 2024) at 32K rendered context on a single RTX PRO 6000 Blackwell GPU, with context-only SnapKV-style compression (applied only to the evictable document context, with the Q_1 prompt and answer tail preserved) as the initial post-Q_1 compressor applied before repair (Li et al., 2024a). The task family is a split-query version of multi-query needle-in-a-haystack (MQ-NIAH) derived from RULER (Hsieh et al., 2024). Each needle binds a key to a value inside a long distractor context, and the model must return the values for the keys requested in the current turn. The 2Q, 4Q, 6Q, and 8Q variants use the same underlying task template. The name denotes how many key/value needles are queried across the two turns. Larger variants ask for more values and use longer decode budgets, but do not change the benchmark family. This synthetic family is designed for control: the future-relevant spans are annotated, the second-turn relevance shift is explicit, and every condition shares the same turn-N history. For each variant, we pool balanced, non-recency-favored

partitions whose Q_2 side excludes the tail-recent needles. Appendix Figure 9 summarizes the exact splits.

Evaluation configuration. The main study avoids both recency saturation and the all-zero baseline regime found at smaller budgets. Unless stated otherwise, the fixed sweep is:

- compressor: context-only SnapKV with $R_{\text{ctx}} = 128$ (SnapKV observation-window length);
- base budgets: $B_{\text{base}} = 8192$ for 2Q, $B_{\text{base}} = 16384$ for 4Q, and $B_{\text{base}} = 18432$ for 6Q/8Q;
- restore budgets: $K = 8, 16, 24, 32, 48, 64, 80, 96, 128$;
- replay: one shared full-cache-generated Q_1 transcript per setting and condition.

This removes turn-N error-cascade differences so that the frontier isolates resumed-cache selection quality at turn $N{+}1$. MQ-NIAH-2Q/4Q/6Q use fixed 100-example evaluation sets per split; MQ-NIAH-8Q uses 24 examples per split over five partitions. Prompt and decode settings are reported in the appendix.

Scorers. The turn-N eviction signal is the exact generated Q_1 answer tail instead of a generic last-32-token cache suffix. For the main quality curves, we append Q_2 to the compressed active cache, extract the model's post-RoPE Q_2 query projections, and score active and offloaded key rows under those projections before selecting evicted rows for promotion. This exact-Q$_2$ selector gives a controlled measurement of how well the newly revealed turn signal identifies rows worth promoting.

Baselines and references. Every condition reuses the same full-cache-generated turn-N transcript when answering Q_2. We compare the full-cache reference A, the base compressed cache B (the post-Q_1 compressed state before any repair; distinct from the budget B_{base}), matched no-repair B_{match}, Random-K and Oldest-K restore controls, and REPAIRKV. Random-K and Oldest-K promote from the same evicted-KV store as REPAIRKV without future-query scoring, so they test whether gains come from query-aware repair or generic reinsertion. The specificity study additionally uses StaleQ-K (score with the previous-turn query), WrongQ-K (score with another example's Q_2), and Refresh-buffered, a stronger Q_2-time reference that reselects the whole $B_{\text{base}} + K$ context budget from active and offloaded rows without full-prefix recomputation. We use Refresh-buffered to expose selector headroom. The main claim is that idle-window repair is useful under matched active cache, not that the current selector is optimal among all Q_2-aware reselectors.

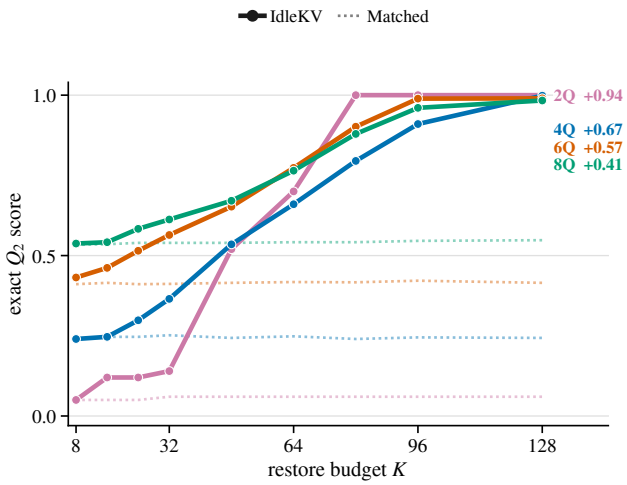


Figure 2. Raw exact Q_2 score under matched resumed-cache budgets. Solid curves are REPAIRKV. Dotted curves are matched no-repair. Right labels report query count and Δ_{96}, the score gain over matched no-repair at $K = 96$. Figure 5 gives promoted-span overlap. Appendix Figure 9 gives per-partition SpanRef-K endpoints.

5. Results

Matched-budget frontier. Across the 2Q/4Q/6Q/8Q MQ-NIAH family, REPAIRKV separates from matched no-repair at moderate-to-large restore budgets under the same resumed active-cache budget. At $K = 96$, REPAIRKV scores 0.910 on 4Q, 0.989 on 6Q, and 0.960 on 8Q, while matched no-repair scores 0.245, 0.422, and 0.546, respectively; Figure 2 reports the full raw Q_2 answer-score frontier. Random-K and Oldest-K stay near matched no-repair in the same evaluations, showing that generic reinsertion from the same offloaded evicted-KV store does not explain the effect. MQ-NIAH-2Q saturates by $K = 80$, while matched no-repair remains near 0.06. Appendix diagnostics report SpanRef-K headroom and operating regimes: on MQ-NIAH-6Q at $K = 48$, SpanRef-K scores 0.994 versus 0.653 for REPAIRKV.

Next-turn signal specificity. Repair gains are specific to the newly revealed current-turn signal. At $K = 48$, StaleQ-K and WrongQ-K stay near matched no-repair, with gains 0.028 $[-0.021, 0.083]$ and $[-0.014, 0.069]$, while REPAIRKV reaches 0.326 $[0.243, 0.410]$. Refresh-buffered's perfect score confirms the right tokens persist in the active and offloaded keys under matched conditions, so idle-window repair is feasible in principle. REPAIRKV's gain over matched no-repair is preliminary evidence that even a simple K-promotion selector captures meaningful gains, and designing stronger selectors is an open research direction.

Repeated relevance shifts. The repair effect also survives repeated controlled relevance shifts. On a fixed five-turn

Table 1. Next-turn signal specificity at $K = 48$ on MQ-NIAH-4Q (72 paired example-partition cases). Stale and donor controls test whether any query-like signal suffices; Refresh is a full-budget reselector over active and offloaded KV.

Condition	Q_2 score	Δ vs. matched [95% CI]
Matched no-repair	0.215	–
StaleQ-K	0.243	+0.028 [−0.021, 0.083]
WrongQ-K	0.243	+0.028 [−0.014, 0.069]
RepairKV	0.542	+0.326 [0.243, 0.410]
Refresh-buffered	1.000	+0.785 [0.722, 0.847]

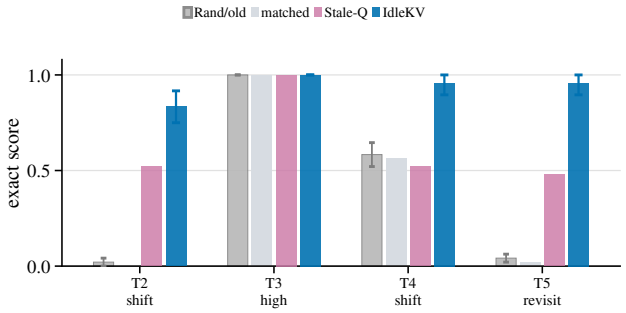


Figure 3. Five-turn relevance-shift diagnostic on MQ-NIAH-8Q (B_{base}=18,432, K=80, n=24). Groups show post-priming exact scores at equal active-cache budget. Intervals are bootstrap 95% CIs for RepairKV, and Stale-Q uses the previous-turn query.

MQ-NIAH-8Q revisit schedule ($K = 80$, $n = 24$), RepairKV gains 0.542 over matched no-repair on non-initial turns with paired bootstrap interval $[0.458, 0.620]$. Random-K and Oldest-K gain only 0.010 and 0.021. Stale-query repair remains below current-query repair, supporting repeated cue-conditioned adaptation under controlled relevance shifts. End-to-end task validation requires trace-based agent benchmarks.

Eviction-policy sensitivity. The adaptation effect persists under two protocol-matched compression policies beyond the primary SnapKV-style policy. The structurally different sink-plus-recent policy (StreamingLLM-style: preserve initial sink tokens plus a recent window (Xiao et al., 2024)) also passes the gate: at $K = 128$, RepairKV gains 0.431 over matched no-repair while Random-K/Oldest-K remain at the matched baseline. These are protocol-matched first-stage eviction policies inspired by H_2O and StreamingLLM; canonical reproductions would require the full original online policies. A separate Scissorhands-style persistence stress test (Liu et al., 2023) on MQ-NIAH-6Q also passes the locked gate (Appendix Table 3).

Robustness and mechanism. At $K = 128$, all twelve balanced partitions pass the direction check: RepairKV beats B_{match} and both content-agnostic restore controls, with minimum gains 0.585/0.387/0.292 for 4Q/6Q/8Q. Appendix

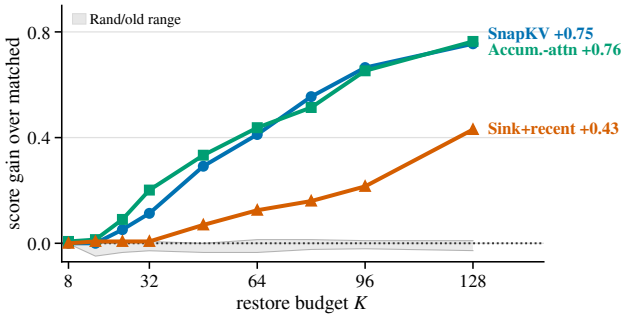


Figure 4. First-stage eviction-policy sensitivity on MQ-NIAH-4Q (n=24, B_{base}=16,384). Curves show exact Q_2 gain over each policy's matched no-repair baseline. The gray band spans Random-K/Oldest-K gains.

Figure 5. Mechanism diagnostic for the final exact MQ-NIAH evaluations. Curves show overlap between promoted tokens and annotated Q_2 answer spans. Content-agnostic promotions stay near baseline while RepairKV increasingly covers future-relevant spans.

Figure 9 gives the per-partition endpoints. Figure 5 shows that RepairKV overlap with annotated Q_2 answer spans rises with K, while controls stay near baseline.

Real-repository diagnostic. To test external validity beyond MQ-NIAH, we ran a controlled real-repository relevance-shift diagnostic: 48 callsite examples from 12 pinned SWE-bench-pool repositories. Repair sees only an answer-redacted event cue (the callsite description with the target identifier removed), and all conditions decode the same event-plus-question prompt. At $K = 192$, event-only RepairKV improves exact identifier accuracy from 18.8% matched to 72.9%. Non-query controls and ToolFile-K (file-name-assisted with oldest-row backfill) remain near matched. File-gated RepairKV (label-assisted, restricting candidates to the event-named file) reaches 83.3%. Label-assisted AnchorWindow (oracle locality reference) reaches 89.6%. This remains a small external-validity diagnostic, with no claim to be a real-code benchmark. Strict exclusions keep the gain positive. Appendix Figure 7 gives the full audit.

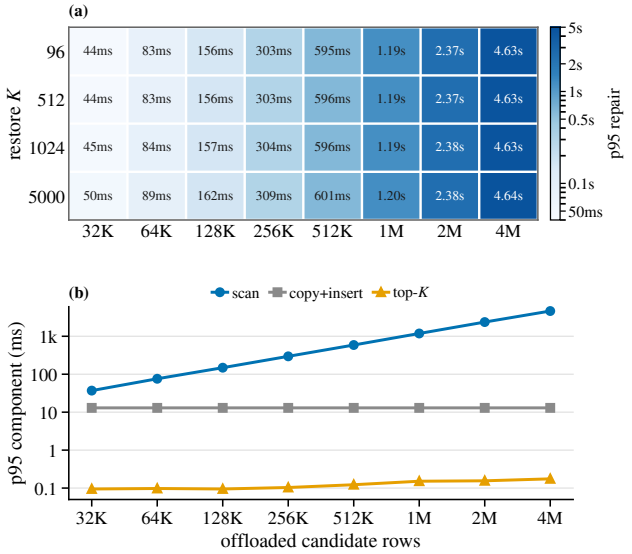


Figure 6. Runtime-capacity probe on the evaluation GPU. (a) Component-summed p95 service time for chunked scan, top-K selection, and KV movement at query length 64 over pinned host-memory BF16 KV-shaped tensors. (b) Stage-wise p95 component time for $K = 5000$. Generation and answer evaluation are excluded.

Runtime accounting. We run a separate GPU runtime probe for the repair operation. The probe measures chunked scan, top-K selection, and KV movement over pinned host-memory BF16 tensors with Qwen-shaped dimensions. Figure 6 reports the resulting capacity envelope, giving a controlled estimate of how repair time scales with the number of offloaded candidate rows and the restore budget.

Capacity envelope. At $K = 5000$, the p95 repair envelope grows from 50.0 ms at 32K offloaded candidates to 1.20 s at 1M and 4.64 s at 4M. These measurements show the operating range for a single-node GPU implementation; trace-scheduled serving latency is a natural next systems step.

Table 2. Preliminary real-repository diagnostic at $K = 192$ over 48 callsite cases from 12 pinned SWE-bench-pool repositories. Accuracy is exact target-identifier recovery. Non-query summarizes the best of random, oldest, stale-cue, and wrong-event controls. AnchorWindow and File-gated are label-assisted; non-label methods are Matched, RepairKV, and ToolFile-K.

Condition	Accuracy
Matched no-repair	18.8%
Best non-query control	16.7%
ToolFile-K	18.8%
RepairKV	72.9%
File-gated RepairKV	83.3%
AnchorWindow-K	89.6%

6. Discussion and Limitations

RepairKV is a between-turn maintenance primitive for making KV compression revisable: a runtime adaptation operator over active KV state for resource-adaptive tiered-KV runtimes. It complements existing compression, offload, query-aware access, and pause/resume serving systems (Kwon et al., 2023; Abhyankar et al., 2024; Liu et al., 2025; Dzikanyanga et al., 2026; Li et al., 2026; Zhang et al., 2026) by asking how active memory can be adapted after compression but before resumed decoding. The mechanism should be most useful when:

- the next turn changes which old context matters;
- the compressed cache still leaves useful choices in the evicted host-memory store; and
- the idle window or scheduler has enough slack for scoring and KV movement.

The experiments isolate one paused request on one GPU, matching local or dedicated agent inference but omitting multi-tenant contention. In shared serving, a scheduler would still need to account for HBM blocks, copy bandwidth, and queueing slack. The quality comparisons match active GPU KV budget; total host-plus-device storage and end-to-end service cost are reported separately. The exact-Q_2 selector provides a controlled view of the repair signal, and stronger Q_2-aware reselectors, page-level policies, or full-prefix recompute may further improve the quality-runtime frontier when enough idle-window slack is available. The real-repository diagnostic is small and exploratory, so it supports external validity without serving as a SWE-bench issue-resolution benchmark. Next experiments should focus on trace-scheduled systems repair with page-level or learned selectors, plus stricter evaluation with time/compute-matched recompute baselines.

Conclusion. RepairKV makes the case for revisable KV compression. If context relevance changes after eviction, the cache lifecycle can move in both directions: compress to fit the active budget, then repair by promoting relevant evicted state before the next decode. Across MQ-NIAH-4Q/6Q/8Q, this repair beats matched no-repair while random and oldest-token promotions stay near baseline. The broader direction is mutable KV memory for resource-adaptive long-context inference, with pause windows treated as part of the inference budget.

References

Abhyankar, R., He, Z., Srivatsa, V., Zhang, H., and Zhang, Y. InferCept: Efficient intercept support for augmented large language model inference. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., Dong, Y., Tang, J., and Li, J. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

Bian, S., Yan, M., Jayarajan, A., Pekhimenko, G., and Venkataraman, S. What limits agentic systems efficiency. In *NeurIPS 2025 Workshop on Scaling Environments for Agents (SEA)*, 2025.

Bian, Z., Wu, F., Ma, T., and Zhuo, Y. Tokencake: A KV-cache-centric serving framework for LLM-based multi-agent applications. *arXiv:2510.18586*, 2025.

Chen, A., Geh, R., Grover, A., Van den Broeck, G., and Israel, D. The pitfalls of KV cache compression. *arXiv:2510.00231*, 2025.

Dzikanyanga, G., Yang, W., Huang, H., Wu, D., Wang, S., Xia, W., and K C, S. TTKV: Temporal-tiered KV cache for long-context LLM inference. *arXiv:2604.19769*, 2026.

Gao, B., He, Z., Sharma, P., Kang, Q., Jevdjic, D., Zhao, J., and Zuo, P. Cost-efficient large language model serving for multi-turn conversations with CachedAttention. In *Proceedings of the 2024 USENIX Annual Technical Conference (USENIX ATC)*, 2024.

Hsieh, C.-Y., Sun, S., Kriman, S., Acharya, S., Rekesh, D., Jia, F., Zhang, Y., and Ginsburg, B. RULER: What's the real context window size of your LLM. In *Conference on Language Modeling (COLM)*, 2024.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. SWE-bench: Can language models resolve real-world GitHub issues. In *International Conference on Learning Representations (ICLR)*, 2024.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H., Cai, T., Lewis, P., and Chen, D. SnapKV: LLM knows what you are looking for before generation. *arXiv:2404.14469*, 2024.

Li, Z., Yang, Y., Zhang, Y., and Liu, Z. PyramidKV: Dynamic KV cache compression based on pyramidal information funneling. *arXiv:2406.02069*, 2024.

Li, Y., Jiang, H., Wu, Q., Luo, X., Ahn, S., Zhang, C., Abdi, A. H., Li, D., Gao, J., Yang, Y., and Qiu, L. SCBench: A KV cache-centric analysis of long-context methods. In *International Conference on Learning Representations (ICLR)*, 2025.

Li, H., Mang, Q., He, R., Zhang, Q., Mao, H., Chen, X., Zhou, H., Cheung, A., Gonzalez, J., and Stoica, I. Continuum: Efficient and robust multi-turn LLM agent scheduling with KV cache time-to-live. *arXiv:2511.02230*, 2026.

Liu, Y., Cheng, Y., Yao, J., An, Y., Chen, X., Feng, S., Huang, Y., Shen, S., Zhang, R., Du, K., and Jiang, J. LMCache: An efficient KV cache layer for enterprise-scale LLM inference. *arXiv:2510.09665*, 2025.

Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., Gu, Y., Ding, H., Men, K., Yang, K., Zhang, S., Deng, X., and others. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR)*, 2024.

Liu, Z., Desai, A., Liao, F., Wang, W., Xie, V., Xu, Z., Kyrillidis, A., and Shrivastava, A. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., Zhao, S., Hong, L., and others. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations (ICLR)*, 2024.

Qwen Team. Qwen2.5 technical report. *arXiv:2412.15115*, 2024.

Pagonas, N., Chung, Y., Kaffes, K., and Krishnamurthy, A. Cortex: Workflow-aware resource pooling and scheduling for agentic serving. *arXiv:2510.14126*, 2025.

Qi, Y., Chen, X., Jiang, H., Wang, Q., Peng, B., and Palpanas, T. ParisKV: Fast and drift-robust KV-cache retrieval for long-context LLMs. *arXiv:2602.07721*, 2026.

Wu, W., Wang, Y., Xiao, G., Peng, H., and Fu, Y. Retrieval heads mechanistically explain long-context factuality. *arXiv:2404.15574*, 2024.

Sun, H., Chang, L.-W., Bao, W., Zheng, S., Zheng, N., Liu, X., Dong, H., Chi, Y., and Chen, B. ShadowKV: KV cache in shadows for high-throughput long-context LLM inference. *arXiv:2410.21465*, 2025.

Tang, J., Zhao, Y., Zhu, K., Xiao, G., Kasikci, B., and Han, S. QUEST: Query-aware sparsity for efficient long-context LLM inference. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

Wang, D., Liu, Z., Wang, S., Ren, Y., Deng, J., Hu, J., Chen, T., and Yang, H. FIER: Fine-grained and efficient KV cache retrieval for long-context LLM inference. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025.

Xiao, G., Tang, Y., Zuo, J., Guo, J., Yang, S., Tang, H., Fu, Y., and Han, S. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.

Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C., Wang, Z., and Chen, B. H_2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Zhang, X., Chen, Y., Hu, S., Xu, Z., Chen, J., Hao, M. K., Han, X., Thai, Z. L., Wang, S., Liu, Z., and Sun, M. ∞Bench: Extending long context evaluation beyond 100K tokens. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

Zhang, H., Zhang, H., Ma, X., Zhang, J., and Guo, S. LazyEviction: Lagged KV eviction with attention pattern observation for efficient long reasoning. *arXiv:2506.15969*, 2025.

Zhang, Y., Yan, Y., Yang, N., and Yuan, D. AgentServe: Algorithm-system co-design for efficient agentic AI serving on a consumer-grade GPU. *arXiv:2603.10342*, 2026.

Zheng, Y., Fan, J., Fu, Q., Yang, Y., Zhang, W., and Quinn, A. AgentCgroup: Understanding and controlling OS resources of AI agents. *arXiv:2602.09345*, 2026.

Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., Alon, U., and Neubig, G. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations (ICLR)*, 2024.

A. Additional Evaluation Details

Protocol details. Concise split prompts request values only, comma-separated, with a short answer prefix and fixed decode budgets of 24 tokens for MQ-NIAH-4Q, 48 tokens for MQ-NIAH-6Q, and 64 tokens for MQ-NIAH-8Q. Generation is deterministic greedy decoding with no sampling. The main examples use dataset seed offset 0. Random-K uses a deterministic example- and K-dependent seed. Bootstrap intervals are percentile intervals over paired example-partition rows with fixed seeds. Main Qwen runs use BF16 weights on one RTX PRO 6000 Blackwell GPU with Python 3.12.3, PyTorch 2.10.0+cu128, and Transformers 5.2.0. The exact-Q_2 attention-query selector appends Q_2 to the compressed active cache, extracts the model's post-RoPE query projections for the Q_2 prompt tokens, and scores active and offloaded key rows under those projections before selecting evicted rows for promotion. This selector is distinct from decode-time answer-token attention and from the final answer metric. The Q_2-appended-state proxy appends Q_2 to the compressed cache and uses the appended cache rows as a second Q_2-conditioned scoring signal.

Span-group diagnostic. SpanRef-K starts from the same B state and offloaded evicted-KV store as REPAIRKV. It enumerates the small family of annotated Q_2 answer-span-group subsets, one key/value span group per requested needle, evaluates each subset once under actual Q_2 generation, and chooses the highest-scoring subset with total cost at most K. It is exact over this benchmark-specific span-group family, but not over all arbitrary token subsets or possible final generations. Because the selected span-group subset may use fewer than K rows, a repair selector can occasionally score higher at low budgets by spending the full budget on useful local neighborhoods around future-relevant spans. Thus SpanRef-K is a diagnostic span-group reference, not a ceiling.

Partition choices. For MQ-NIAH-4Q, the pooled panel uses 14→23, 24→13, and 34→12. These are the balanced 2|2 partitions whose Q_2 side excludes the tail-anchored fourth needle. For MQ-NIAH-6Q, the pooled panel uses 156→234, 256→134, 356→124, and 456→123, so the second turn excludes the two latest needles. For MQ-NIAH-8Q, the pooled panel uses 5678→1234, 1678→2345, 2678→1345, 3678→1245, and 4678→1235, again excluding the two latest needles from the second turn. Within-turn order is not treated as a separate condition because the prompt asks for comma-separated values only and scoring checks whether each requested value is recovered. Recency-favorable splits such as 12→34 are left to separate diagnostics outside the main future-query repair result.

Scissorhands-style persistence stress test. Table 3 repeats the MQ-NIAH-6Q locked protocol after replacing the initial SnapKV-style eviction policy with a one-shot persistence policy inspired by Scissorhands. The test asks whether repair still works when the compressed active cache is chosen by past attention persistence in place of the primary eviction policy. Canonical online per-head Scissorhands reproduction remains future work.

Table 3. Scissorhands-style persistence stress test on MQ-NIAH-6Q (B_{base}=18,432, n=24). Values are exact scores. Best ctrl. is the stronger of Random-K and Oldest-K; the interval is the paired bootstrap 95% CI for REPAIRKV$-B_{\text{match}}$.

K	Full	B_{match}	Best ctrl.	REPAIRKV	Gain CI
48	0.990	0.365	0.382	0.712	[0.292, 0.403]
64	0.990	0.365	0.382	0.795	[0.378, 0.490]
80	0.990	0.378	0.385	0.934	[0.503, 0.604]
96	0.990	0.375	0.382	0.993	[0.566, 0.670]
128	0.990	0.368	0.378	0.990	[0.573, 0.674]

B. Additional Discussion

Tiered-cache scaling. Repair creates a two-level cache-budget problem because the main curves match the resumed active GPU KV budget while tracking total host-plus-device storage separately. A strict device-resident policy chooses the active-cache budget, while a looser off-device policy decides which evicted rows remain searchable, compressed, summarized, or recomputable. In the Qwen-shaped accounting above, 4M searchable rows already imply about 240 GB before metadata. Thus REPAIRKV supports the middle operation in a tiered hierarchy: turn-conditioned promotion from host memory during an idle window. This is a hardware-facing implication; hardware measurement remains future work.

Idle-window repair interface. A practical implementation would expose a host-memory index over evicted KV units, a time-budgeted selector conditioned on the next-turn signal, asynchronous promotion into the active cache, and fallbacks to

no-repair or full recompute. This turns a pause into a schedulable resource: the system can spend only the slack that remains after queueing, transfer, and generation constraints, and can move the selector from token rows to pages or compressed metadata as hardware and serving constraints require.

Future benchmark axes. Future benchmarks should vary the number, timing, and conflict structure of new-turn constraints. The 2Q and 8Q breadth checks suggest that easy regimes can saturate while stricter multi-query turns expose how quickly eviction failures accumulate. The five-turn diagnostic strengthens the dynamic-workflow evidence, but it remains synthetic. Broader task families should keep the same matched-budget controls while adding real tool traces and stronger selector families.

C. Supplementary Experimental Views

The supplementary figures probe external validity, query-count and operating-regime breadth, partition consistency, runtime capacity, and appendix selector diagnostics. Exact audit values remain in released result artifacts.

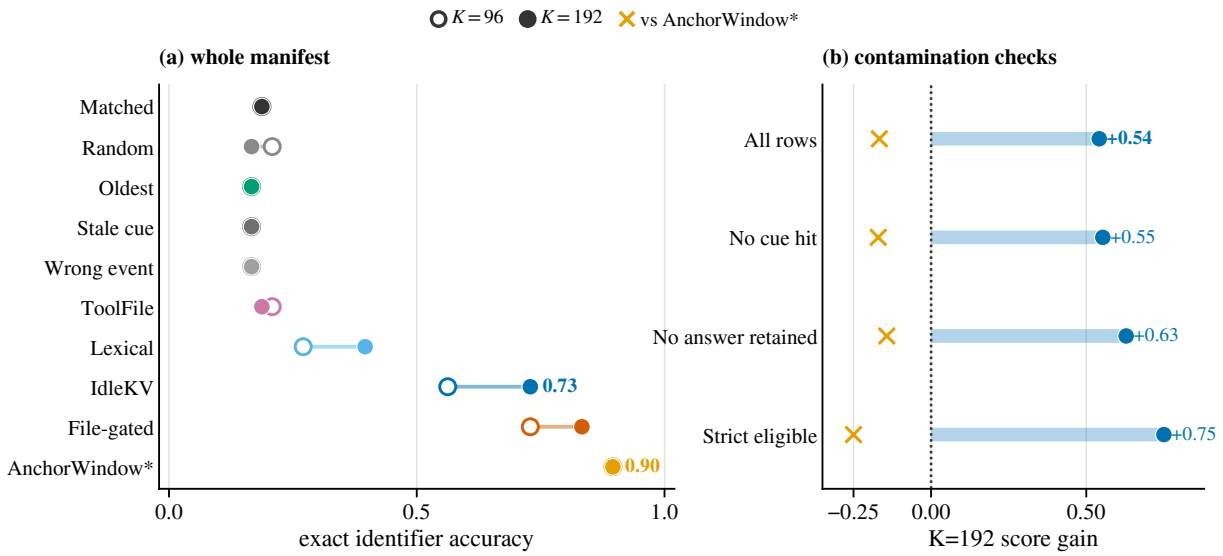
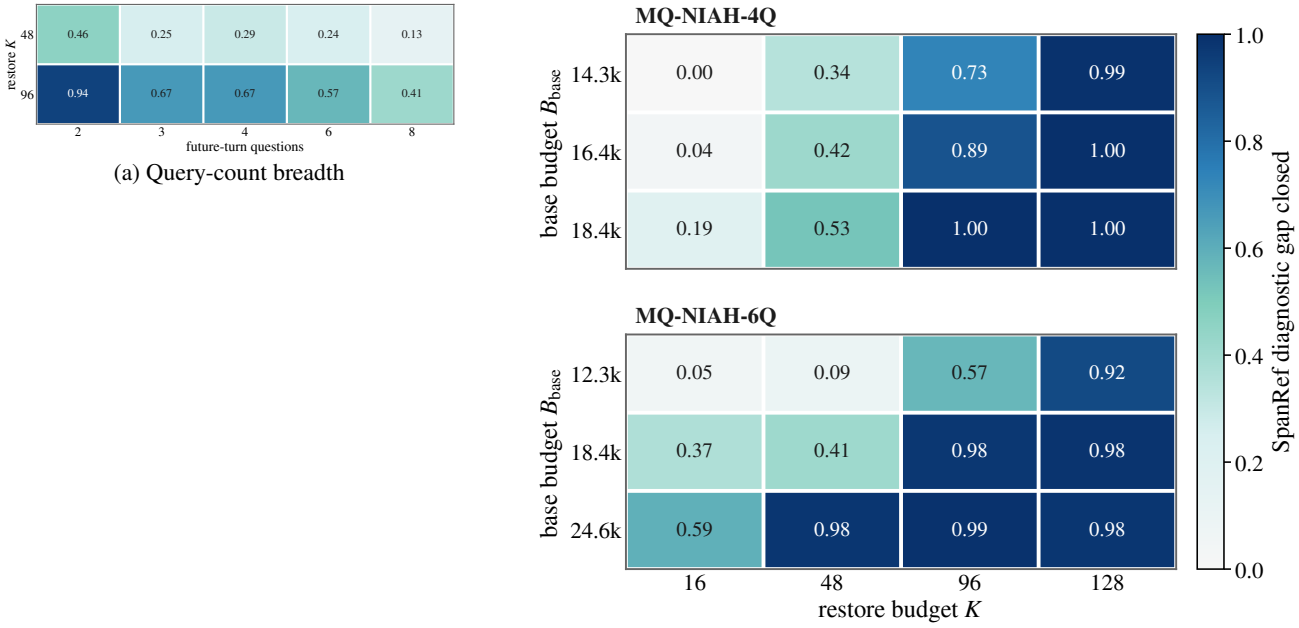


Figure 7. Controlled real-repository relevance-shift diagnostic over 48 callsite examples from 12 repositories drawn from the SWE-bench repository pool at pinned commits. Repair sees only the event cue. All conditions decode the same event-plus-question prompt. Left: exact identifier accuracy at $K = 96$ and $K = 192$. Right: $K = 192$ REPAIRKV gain over matched no-repair after removing cue-only or answer-retention rows. ToolFile-K is file-name-assisted with oldest-row backfill. File-gated REPAIRKV restricts candidates to the event-named file before applying the event-only scorer. Lexical is an answer-sanitized diagnostic outside the deployable-selector comparison. AnchorWindow-K is a label-assisted locality reference outside the deployable-baseline and end-to-end issue-resolution comparisons.



(a) Query-count breadth

(b) Operating regime

Figure 8. Supplementary breadth diagnostics. (a) Cells show REPAIRKV score gain over matched no-repair for $K = 48$ and $K = 96$ across 2-, 3-, 4-, 6-, and 8-question split-query variants. (b) Cells show the fraction of available SpanRef-K diagnostic gain recovered by REPAIRKV, clipped to $[0, 1]$, for MQ-NIAH-4Q/6Q. Saturated cells with little matched-to-SpanRef-K gap should not be read as upper-bound statements.

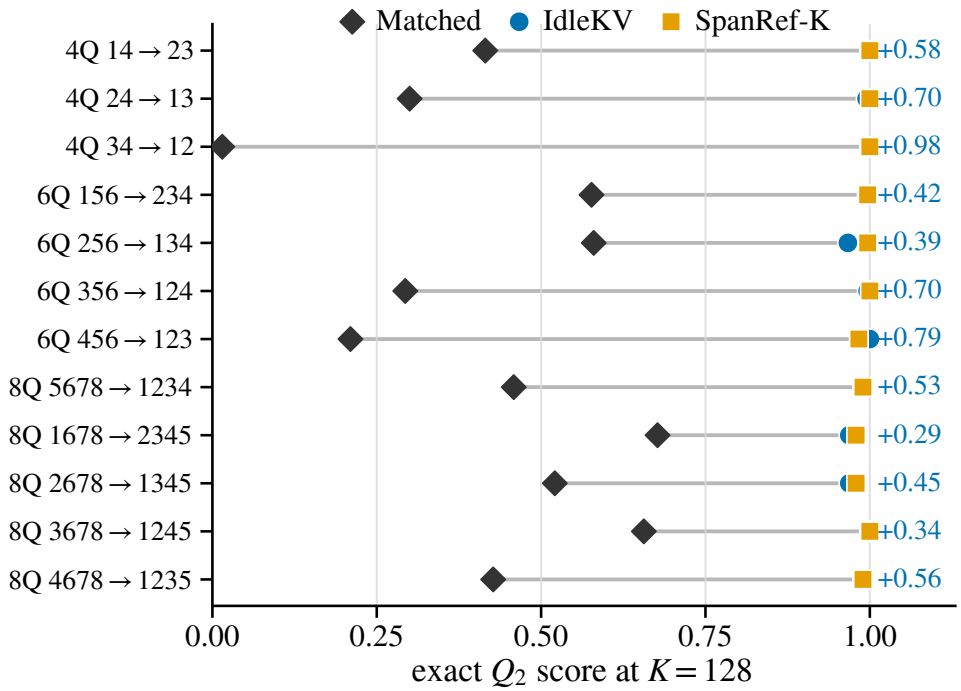
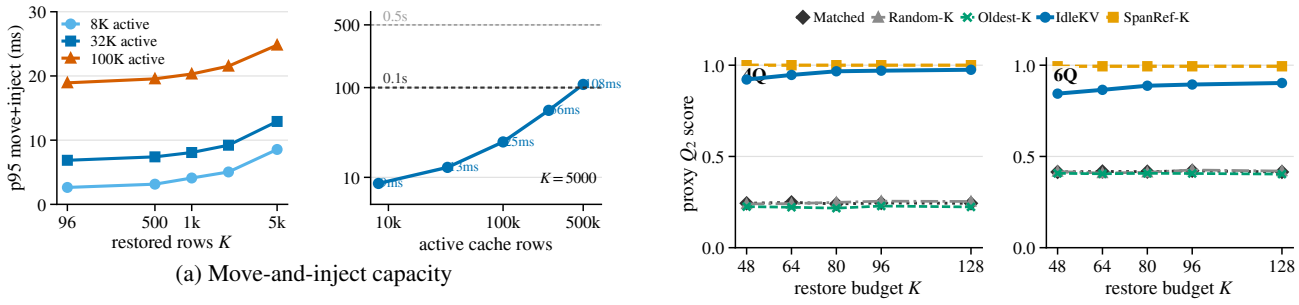


Figure 9. Per-partition endpoint scores at $K = 128$ on the final exact evaluations. Gray segments show the change from matched no-repair to REPAIRKV, orange points show the benchmark-metadata SpanRef-K diagnostic, and labels report the REPAIRKV difference over matched no-repair.



(a) Move-and-inject capacity

(b) Proxy-selector diagnostic

Figure 10. Supplementary runtime and selector diagnostics. (a) Synthetic move-and-inject capacity for Qwen-shaped BF16 KV on the evaluation GPU, separate from the Figure 6 scan/select envelope. Measurements use pinned host-memory promoted blocks and report p95 latency as K or active cache size grows. (b) Appendix proxy-selector diagnostic on MQ-NIAH-4Q/6Q (100 examples per split). At $K = 96$, content-agnostic promotions stay within 0.010/0.004 of matched no-repair while proxy REPAIRKV remains well above matched.